

A Artifact Appendix

A.1 Abstract

This artifact supports two experiments for the paper “Personalized Federated Hyperdimensional Computing via Shared–Personal Prototype Memory Decomposition”: (1) controlled-system reproduction of Tables I, II, and III, and (2) a six-dataset benchmark comparing HoRU, HyperFeel, and FedHDC. The table experiment generates its controlled fixture locally and produces the latency and payload tables. The benchmark experiment acquires public inputs, builds local caches, runs the shared protocol, and writes validation reports.

A.2 Badges sought

We submit this artifact for the following ACM badges:

- Artifacts Available
- Artifacts Evaluated – Functional
- Results Validated – Reproduced

A.3 Artifact check-list (meta-information)

Obligatory. Use just a few informal keywords in all fields applicable to your artifacts and remove the rest. This information is needed to find appropriate reviewers and gradually unify artifact meta information in Digital Libraries.

- **Algorithm:** personalized federated HDC, low-rank shared/personal prototype decomposition
- **Program:** Python package and command-line workflows
- **Compilation:** none; interpreted Python
- **Binary:** optional CUDA-enabled PyTorch
- **Model:** prototype memories and low-rank bases built at run time
- **Data set:** controlled-system fixture, UCI-HAR, ISOLET, FEMNIST, WISDM, LEAF Synthetic, NinaPro DB1
- **Run-time environment:** Python 3.11+, Linux
- **Hardware:** CPU for Tables I–III; one CUDA GPU recommended for the full benchmark
- **Execution:** command line
- **Metrics:** bootstrap latency, recurring-round latency, upload payload, round-25 accuracy
- **Output:** CSV, JSON, Markdown reports
- **Experiments:** table reproduction; six-dataset benchmark and validation
- **How much disk space required (approximately)?:** table workflow under 1 GB; benchmark inputs and outputs require tens of GB
- **How much time is needed to prepare workflow (approximately)?:** a few minutes for the Python environment; dataset acquisition depends on downloads
- **How much time is needed to complete experiments (approximately)?:** tables in minutes; quick benchmark in tens of minutes; full benchmark is GPU-dependent and longer

- **Publicly available?:** yes
- **Code licenses (if publicly available)?:** repository terms
- **Data licenses (if publicly available)?:** upstream dataset licenses
- **Workflow framework used?:** none
- **Archived (provide DOI)?:** none assigned in this repository snapshot

A.4 Description

A.4.1 How to access

The repository root contains the artifact. The active code paths are:

- `artifact/scripts/run_tables123.py` for Tables I–III;
- `artifact/scripts/prepare_and_run_cuda_reconstruction_sequential.py` for the dataset-by-dataset six-benchmark workflow;
- `artifact/scripts/prepare_and_run_cuda_reconstruction_suite.py` for the full six-dataset wrapper;
- `artifact/scripts/run_reconstruction_6x3.py` and `run_hd_checkpoint_comparison.py` for the core HD benchmark endpoints;
- `our_hd/` for the HoRU, HyperFeel, and FedHDC implementations;
- `reference_results/cuda_suite/` for immutable reference reports;
- `configs/dataset_specs.json` for dataset contracts.

A.4.2 Hardware dependencies

The table workflow is CPU-only. The six-dataset reference reports were generated with CUDA; reproducing the full benchmark is practical with one CUDA-capable GPU. The code still runs on CPU, but the full benchmark may be slow.

A.4.3 Software dependencies

Python 3.11 or newer is required. The package dependencies are `torch >= 2.2`, `PyYAML >= 6.0`, and `scipy >= 1.11`. Tests use `pytest >= 8`.

A.4.4 Data sets

Tables I–III use a generated controlled-system fixture and do not require an external dataset. The benchmark uses six public datasets with pinned input-acquisition scripts: UCI-HAR, ISOLET, FEMNIST, WISDM, Synthetic, and NinaPro DB1. Short dataset notes are under `docs/datasets/`.

A.4.5 Models

No pretrained neural network is required. The artifact constructs prototype memories, shared bases, and personal bases during execution.

A.5 Installation

Create a Python environment and install the package:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -e '.[test]'
```

For CUDA benchmark runs, install the appropriate CUDA-enabled PyTorch wheel before the editable install.

The artifact also ships a Docker build path for reviewers:

```
docker build -t horu-ae:1.0 .
```

If a published container image is later attached to the submission release, reviewers may pull it instead of rebuilding locally:

```
docker pull ghcr.io/longnew/horu-artifact:1.0
```

A.6 Experiment workflow

Tables I, II, and III. Generate the controlled-system fixture, then run the table workflow:

```
python3 artifact/scripts/run_tables123.py \
  --data-root data \
  --output-dir results/table_reproduction \
  --warmup 5 \
  --repeats 30 \
  --threads 1
```

The same workflow can be executed in Docker:

```
docker run --rm \
  -v "$PWD/data:/artifact/data" \
  -v "$PWD/results:/artifact/results" \
  horu-ae:1.0 \
  bash scripts/run_tables.sh
```

Six-dataset benchmark. The recommended reviewer path is the sequential wrapper that acquires or generates each dataset and then runs HoRU, HyperFeel, and FedHDC one dataset at a time:

```
python3 artifact/scripts/prepare_and_run_cuda_reconstruction_sequential.py \
  --uci-har-source-root /data/horu/uci_har \
  --uci-har-archive /data/downloads/UCI_HAR_Dataset.zip \
  --isolet-raw-source-root /data/horu/isolet \
  --isolet-download-dir /data/downloads/isolet \
  --femnist-source-root /data/horu/femnist \
  --wisdm-source-root /data/horu/wisdm \
  --wisdm-outer-archive /data/downloads/WISDM.zip \
```

```
--synthetic-source-root /data/horu/synthetic \
--ninapro-db1-source-root /data/horu/ninapro \
--ninapro-download-dir /data/downloads/ninapro \
--output-dir results_reconstruction/cuda_suite_sequential
```

This wrapper acquires the public inputs, builds pinned local caches, writes one subdirectory per dataset, and emits a top-level `sequential_summary.json`. The all-at-once wrapper remains available as `artifact/scripts/prepare_and_run_cuda_reconstruction_suite.py`.

Inside Docker on a GPU-capable host, reviewers can run:

```
docker run --rm --gpus all \
-v "$PWD/data:/artifact/data" \
-v "$PWD/results:/artifact/results" \
horu-ae:1.0 \
bash scripts/run_benchmark_sequential.sh
```

A.7 Evaluation and expected results

Tables I, II, and III. The table workflow must generate:

- `table1.csv`
- `table2.csv`
- `table3.csv`
- `raw_timings.csv`
- `environment.json`
- `result.json`

The expected qualitative outcome is that HoRU shows lower recurring-round latency and much smaller upload payload than FedHDC and HyperFeel.

Six-dataset benchmark. The immutable reference report is under `reference_results/cuda_suite/`. It can be checked with:

```
bash scripts/verify_reference.sh
```

The expected round-25 accuracies in the committed reference reports are:

- UCI-HAR: HoRU 97.66, HyperFeel 98.08, FedHDC 95.25
- ISOLET: HoRU 88.14, HyperFeel 90.59, FedHDC 93.85
- FEMNIST: HoRU 68.54, HyperFeel 67.78, FedHDC 57.26
- WISDM: HoRU 56.69, HyperFeel 51.11, FedHDC 6.95
- Synthetic: HoRU 78.00, HyperFeel 73.19, FedHDC 71.09
- NinaPro DB1: HoRU 75.68, HyperFeel 61.48, FedHDC 30.86
- Unweighted mean: HoRU 77.45, HyperFeel 73.71, FedHDC 59.21

A.8 Experiment customization

The intended customization surface is narrow:

- choose a different output directory;
- run only a subset of datasets via repeated `--dataset` flags;
- point dataset-root and download-root flags at existing local caches.

Reviewers should not edit the committed manifest or metric contract when the goal is to validate the reference results. If GPU resources are unavailable, the table workflow remains fully reproducible on CPU.